

NLP et Statistiques

One-Hot Encoding (OHE)

- One-Hot Encoding (OHE)
- Le One-Hot Encoding représente chaque mot comme un vecteur unique où une seule position vaut 1 et toutes les autres 0. C'est une représentation au niveau du mot.
- Usage : Utilisé principalement dans les réseaux de neurones (Deep Learning) avant les couches d'Embedding.
- Inconvénient : Les vecteurs deviennent gigantesques et "vides" (sparse) si le vocabulaire est large.

One-Hot Encoding (OHE)

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

mots = np.array(['chat', 'souris', 'fromage', 'chat']).reshape(-1, 1)
ohe = OneHotEncoder(sparse_output=False)

# Chaque mot unique reçoit un vecteur binaire propre
print(ohe.fit_transform(mots))
```


One-Hot Encoding (OHE)

One-Hot

Le Mot

Un vecteur binaire par mot.

Bag of Words

Le Document

Un vecteur de fréquences pour toute la phrase.

CountVectorizer

CountVectorizer est un outil de la bibliothèque Scikit-Learn (Sklearn) utilisé en Traitement du Langage Naturel (NLP) pour transformer des textes en données numériques.

Il repose sur le modèle du sac de mots (**Bag of Words**) : chaque document est représenté par un vecteur indiquant la fréquence d'apparition de chaque mot du vocabulaire global.

Pour l'utiliser, il faut importer la classe depuis le module `sklearn.feature_extraction.text`

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
# 1. Création d'un petit corpus en français
```

```
corpus = [  
    "Le chat mange la souris.",  
    "La souris mange le fromage.",  
    "Le chat court après la souris."  
]
```

```
# 2. Initialisation de l'outil
```

```
vectorizer = CountVectorizer()
```

```
# 3. Apprentissage du vocabulaire et transformation en matrice
```

```
X = vectorizer.fit_transform(corpus)
```

```
# 4. Affichage du vocabulaire indexé
```

```
print(vectorizer.get_feature_names_out())
```

```
# Résultat : ['après', 'chat', 'court', 'fromage', 'la', 'le', 'mange', 'souris']
```

```
# 5. Affichage de la matrice (fréquences)
```

```
print(X.toarray())
```


Pour améliorer la précision sur des textes vous pouvez configurer plusieurs paramètres lors de l'initialisation :

stop_words : Permet d'ignorer les mots très communs (le, la, de, etc.) qui n'apportent pas de sens sémantique. Bien que Scikit-Learn n'ait pas de liste "french" intégrée par défaut comme pour l'anglais, vous pouvez fournir votre propre liste.

ngram_range : Permet de capturer des groupes de mots (ex: (1, 2) pour capturer "chat" et "le chat").

lowercase : Convertit tout le texte en minuscules par défaut (True).

max_df / min_df : Filtre les mots trop fréquents ou trop rares.

- **Tokenisation** : Le texte est découpé en unités individuelles (mots).
- **Construction du vocabulaire** : L'outil identifie tous les mots uniques présents dans l'ensemble des documents.
- **Encodage** : Chaque phrase devient un vecteur de nombres où chaque position correspond à un mot du vocabulaire

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

corpus = ["Le chat mange la souris.", "La souris mange le fromage."]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

# Création du DataFrame avec les mots en colonnes
df = pd.DataFrame(
    data=X.toarray(),
    columns=vectorizer.get_feature_names_out(),
    index=['Doc 1', 'Doc 2']
)
```

Résultat :

	chat	fromage	la	le	mange	souris
Doc 1	1	0	1	1	1	1
Doc 2	0	1	1	1	1	1

TF-IDF (Term Frequency - Inverse Document Frequency)

- Le TF-IDF (Term Frequency - Inverse Document Frequency) est une mesure statistique évaluant l'importance d'un terme dans un document au sein d'un corpus. Il multiplie la fréquence du mot dans le document (TF) par l'inverse de sa fréquence dans l'ensemble des documents (IDF), permettant d'identifier les mots-clés pertinents en écartant les termes trop courants.

TF-IDF (Term Frequency - Inverse Document Frequency)

Term Frequency (TF) - Fréquence du terme

Le TF mesure la fréquence d'apparition d'un mot dans un document spécifique. Plus un mot apparaît souvent, plus son TF est élevé.

$$\text{Formule : TF} = \frac{\text{Nombre d'occurrences du terme dans le document}}{\text{Nombre total de mots dans le document}}.$$

TF-IDF (Term Frequency - Inverse Document Frequency)

Inverse Document Frequency (IDF) - Fréquence inverse du document

L'IDF mesure l'importance d'un terme dans l'ensemble d'un corpus (collection de documents). Il réduit le poids des mots très fréquents (comme "le", "et", "est") et augmente celui des mots rares, jugés plus significatifs

$$\text{Formule : IDF} = \log \left(\frac{\text{Nombre total de documents}}{\text{Nombre de documents contenant le terme}} \right).$$

TF-IDF - Calcul Final

Le score final est le produit de ces deux valeurs :

Principe : Un score TF-IDF élevé est obtenu par une fréquence de terme élevée dans un document donné (TF élevé) et une faible fréquence du terme dans l'ensemble du corpus (IDF élevé).

Utilité : Il est largement utilisé pour le SEO (référencement) afin de mesurer la pertinence d'un contenu et pour la recherche d'information afin de classer les documents par pertinence.

Exemple : Dans un article sur la "pizzas", le mot "pizza" aura un TF élevé, et un IDF moyen (il est courant dans un corpus de cuisine). Le mot "napolitaine" aura un TF moyen mais un IDF très élevé (il est rare dans le corpus), lui donnant un score TF-IDF fort qui le définit comme mot-clé stratégique.

Utilisation de Scikit-learn

Le TfidfVectorizer de Scikit-learn combine le comptage des mots et le calcul du score en une seule étape

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
```

```
# 1. Préparation du corpus (ensemble de documents)
```

```
corpus = [  
    "Le chat mange la souris.",  
    "La souris mange le fromage.",  
    "Le chat dort sur le tapis."  
]
```



```
# 2. Initialisation du vectoriseur
```

```
vectorizer = TfidfVectorizer()
```

```
# 3. Calcul du TF-IDF
```

```
tfidf_matrix = vectorizer.fit_transform(corpus)
```

```
# 4. Affichage des résultats sous forme de tableau
```

```
feature_names = vectorizer.get_feature_names_out()
```

```
df = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)
```

```
print(df)
```

Scikit-learn produit une matrice où chaque mot est une colonne. Les scores sont normalisés (L2) pour que chaque vecteur de document ait une longueur de 1, ce qui explique pourquoi les valeurs sont comprises entre 0 et 1.

Document	chat	dort	fromage	mange	souris	tapis
Doc 1	0.44	0.00	0.00	0.44	0.44	0.00
Doc 2	0.00	0.00	0.58	0.44	0.44	0.00
Doc 3	0.40	0.52	0.00	0.00	0.00	0.52

Note : Des mots comme "mange" et "souris" ont des scores plus faibles dans le Doc 1 car ils apparaissent aussi dans le Doc 2 (IDF plus bas). Le mot "fromage" est unique au Doc 2, il reçoit donc un score plus élevé.

TfidfVectorizer

Il fait tout le travail à partir de votre texte brut.

Entrée : Une liste de chaînes de caractères (phrases brutes).

Action : Il segmente le texte (tokenisation), compte l'occurrence des mots (CountVectorizer) puis applique le calcul du TF-IDF.

Usage : C'est l'outil le plus utilisé quand vous partez de zéro.

TfidfTransformer

Il ne travaille que sur des nombres. Il ne sait pas lire du texte.

Entrée : Une matrice de comptage (générée au préalable par un CountVectorizer).

Action : Il transforme simplement les fréquences brutes (ex: le mot "chat" apparaît 5 fois) en scores TF-IDF pondérés.

Usage : Utile si vous avez déjà une matrice de fréquences ou si vous voulez intégrer le calcul dans un pipeline complexe où l'étape de comptage est séparée.

Caractéristique

TfidfVectorizer

TfidfTransformer

Entrée

Texte brut (["Le chat mange..."])

Matrice de nombres
(Fréquences)

Composition

CountVectorizer +
TfidfTransformer

Uniquement la formule TF-IDF

Rapidité de code

Très rapide (une seule classe)

Nécessite deux étapes


```
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
import pandas as pd
```

```
# 1. Notre corpus de textes
```

```
corpus = [  
    "Le chat mange la souris",  
    "La souris mange le fromage",  
    "Le chat dort sur le tapis"  
]
```

2. ÉTAPE A : CountVectorizer (Compte les occurrences brutes)

Cette étape transforme le texte en nombres

```
vectorizer = CountVectorizer()
```

```
counts = vectorizer.fit_transform(corpus)
```

3. ÉTAPE B : TfidfTransformer (Calcule les poids TF-IDF)

On applique la transformation sur la matrice de nombres précédente

```
transformer = TfidfTransformer()
```

```
tfidf_matrix = transformer.fit_transform(counts)
```

4. Affichage des résultats

```
feature_names = vectorizer.get_feature_names_out()
```

```
df = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)
```

```
print(df)
```

Stopwords

Stopwords Français (Exemples)

- La liste française comprend généralement environ 150 à 160 mots (NLTK). Elle inclut les articles, pronoms, prépositions et auxiliaires

Stopwords

- Articles & Pronoms : le, la, les, un, une, des, ce, cette, ces, mon, ton, son, notre, votre, leur.
- Sujets : je, tu, il, elle, nous, vous, ils, elles, on.
- Prépositions & Conjonctions : de, à, dans, par, pour, avec, sur, sous, chez, et, ou, mais, car.
- Auxiliaires : ai, as, a, avons, avez, ont, suis, es, est, sommes, êtes, sont.
- Adverbes courants : ne, pas, plus, très, déjà, hier.

Stopwords

- Stopwords Anglais (Exemples)
- La liste anglaise de NLTK contient environ 179 mots, tandis que celle de Scikit-learn en compte 318

Stopwords

- Articles & Pronoms : a, an, the, my, your, his, her, its, our, their, this, that, these, those.
- Sujets : I, you, he, she, it, we, they.
- Prépositions & Conjonctions : in, on, at, with, by, for, from, and, but, or, if, because, until, while.
- Auxiliaires & Verbes d'état : am, is, are, was, were, be, been, being, have, has, had, do, does, did.
- Négations & Divers : no, nor, not, only, own, same, so, than, too, very, just, don't, should

Stopwords

```
import nltk
from nltk.corpus import stopwords

# Télécharger les listes si nécessaire
nltk.download('stopwords')

print("Français :", stopwords.words('french'))
print("Anglais :", stopwords.words('english'))
```

Le plongement de mots (ou word embedding en anglais)

Le plongement de mots (ou word embedding en anglais) est l'évolution moderne du TF-IDF. Là où le TF-IDF se contente de compter les mots, le plongement de mots cherche à comprendre leur sens et leurs relations.

C'est quoi concrètement ?

C'est une technique qui transforme chaque mot en un vecteur de nombres (souvent entre 100 et 300 dimensions).

La règle d'or : Deux mots qui ont un sens proche ou qui apparaissent dans des contextes similaires auront des vecteurs très proches géométriquement dans l'espace.

Le plongement de mots (ou word embedding en anglais)

Les modèles célèbres

Word2Vec (Google) : Le pionnier, utilisant des réseaux de neurones simples.

GloVe (Stanford) : Basé sur des statistiques globales de co-occurrence de mots.

FastText (Facebook) : Décompose les mots en sous-mots (ex: "manger", "mangeons", "mangeait"), idéal pour gérer les fautes d'orthographe ou les mots complexes en français.

Le plongement de mots (ou word embedding en anglais)

```
import gensim.downloader as api

# Chargement d'un modèle pré-entraîné (ex: sur Google News)
model = api.load("word2vec-google-news-300")

# Trouver les mots les plus proches de "paris"
result = model.most_similar(positive=['paris', 'germany'], negative=['france'])
print(result[0]) # Devrait afficher 'berlin'
```

Le plongement de mots (ou word embedding en anglais)

Pour obtenir le vecteur (plongement) d'un mot unique, on utilise généralement spaCy (très simple pour le français) ou Gensim.

- Voici comment faire avec spaCy :

Installation

Il faut installer la bibliothèque et le modèle de langue française (le modèle md pour medium contient les vecteurs) :

```
pip install spacy
```

```
python -m spacy download fr_core_news_md
```

Le plongement de mots (ou word embedding en anglais)

```
import spacy

# Charger Le modèle de langue française (medium ou large pour avoir les vecteurs,
nlp = spacy.load("fr_core_news_md")

# Analyser un mot
mot = nlp("café")

# 1. Extraire le vecteur (souvent 300 dimensions)
vecteur = mot.vector

print(f"Dimension du vecteur : {vecteur.shape}")
print(f"Aperçu des 5 premières valeurs : {vecteur[:5]}")

# 2. Bonus : Trouver la similarité avec un autre mot
autre_mot = nlp("thé")
print(f"Similarité entre 'café' et 'thé' : {mot.similarity(autre_mot):.2f}")
```


- Pour calculer la **similarité cosinus** (cosine similarity) entre deux mots, on mesure le cosinus de l'angle entre leurs deux vecteurs de plongement (*embeddings*). Plus la valeur est proche de 1, plus les mots sont sémantiquement proches.

```
import spacy
```

```
# Chargement du modèle français (Le modèle 'md' contient les vecteurs)
```

```
nlp = spacy.load("fr_core_news_md")
```

```
mot1 = nlp("chat")
```

```
mot2 = nlp("chien")
```

```
# Calcul direct de la similarité cosinus
```

```
score = mot1.similarity(mot2)
```

```
print(f"La similarité cosinus entre '{mot1}' et '{mot2}' est : {score:.4f}")
```

```
from sklearn.metrics.pairwise import cosine_similarity
import spacy
import numpy as np

nlp = spacy.load("fr_core_news_md")

# Récupération des vecteurs (embeddings)
vec_pomme = nlp("pomme").vector.reshape(1, -1)
vec_fruit = nlp("fruit").vector.reshape(1, -1)

# Calcul du cosinus
score = cosine_similarity(vec_pomme, vec_fruit)[0][0]

print(f"Score cosinus (manuel) : {score:.4f}")
```


Les bigrammes

- Les bigrammes sont des groupes de deux mots consécutifs extraits d'un texte.
- En NLP, on utilise les N-grammes pour conserver le contexte local. Alors qu'un "unigramme" (un seul mot) perd l'ordre des mots, un bigramme permet de capturer des expressions comme "ne pas" ou "pomme de terre", changeant radicalement le sens par rapport aux mots pris isolément.

Les bigrammes

Utilisation avec CountVectorizer

Dans CountVectorizer, cela se contrôle avec le paramètre `ngram_range=(min, max)`.

(1, 1) : Unigrammes uniquement (par défaut).

(2, 2) : Bigrammes uniquement.

(1, 2) : Unigrammes ET bigrammes (souvent le meilleur choix pour l'analyse).

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd
```

```
corpus = ["Le chat mange la souris."]
```

```
# On configure pour extraire uniquement des bigrammes
```

```
vectorizer = CountVectorizer(ngram_range=(2, 2))
```

```
X = vectorizer.fit_transform(corpus)
```

```
# Affichage du vocabulaire généré
```

```
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
print(df)
```


Résultat produit :

le chat

chat mange

mange la

la souris

1

1

1

1